

THE STREAM CIPHER LEVIATHAN SPECIFICATION AND SUPPORTING DOCUMENTATION

David A. McGrew and Scott R. Fluhrer {mcgrew, sfluhrer}@cisco.com
Cisco Systems, Inc. 170 West Tasman Drive, San Jose, CA 95134

October 24, 2000

Contents

1	Introduction	3
2	Specification	3
2.1	The functions a , b , and c	4
2.2	Key Setup	6
3	Security	7
3.1	Statement on Hidden Weaknesses	7
3.2	Security Analysis	8
3.2.1	Up and Down Attacks	8
3.2.2	Branch Attacks	9
3.2.3	Top Down Attacks	9
3.2.4	Slide Attacks	9
4	Design Rationale	9
5	Performance	12
6	Implementation Notes	13
7	Test Vectors	15

1 Introduction

This paper specifies the synchronous stream cipher LEVIATHAN, and provides the supporting documentation for the NESSIE standardization process. The design of this cipher enables it to efficiently seek to arbitrary locations in its keystream, despite the fact that its state transition functions are nonlinear. The cipher is designed for high throughput on general purpose processors.

2 Specification

LEVIATHAN is a synchronous stream cipher, in which encryption consists of bitwise binary addition of a keystream with the plaintext string. LEVIATHAN is parameterized by the number of bits per output, which is denoted as n , and the number of bytes in the key, which is denoted as m . While the cipher is well defined for all non-zero values of these parameters, and we use this notation in the following, for the purposes of standardization we require that $n = 32$, and we require that $m = 16$ for normal security levels (128 bit keys) and that $m = 32$ for high security levels (256 bit keys).

LEVIATHAN keystream is defined by a set of binary tree structures. Each tree has height 16, and thus has 2^{16} leaves. Each leaf is associated with a single n -bit word of the keystream. There are 2^{32} such trees, thus giving $n2^{48}$ bits of output.

Each node of each tree has $3n$ bits of internal state, and the node state is represented by the variable s . We sometimes use the *decatenated representation* of node state, in which the $3n$ bit variable s is written as $z \mid y \mid x$, which means that the most significant n bits of s have the value z , the next least significant n bits of s have the value y , and the least significant n bits of s have the value x . Here and below, x , y and z are n -bit words. All variables are considered as bit strings, or equivalently, as unsigned integers in binary representation with the least significant bit on the right.

The outputs of LEVIATHAN are n -bit words. The i^{th} word is produced by the following algorithm:

```

set  $r$  to  $i \bmod 2^{16}$ , and set  $s$  to  $1 \mid 0 \mid i/2^{16}$ 
for  $j$  from 15 down to zero do
  if the  $j^{th}$  least significant bit of  $r$  is zero then
    set  $s$  to  $a(s)$ 
  else
    set  $s$  to  $b(s)$ 

```

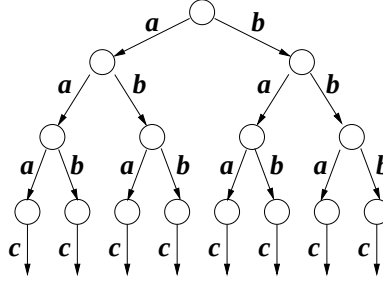


Figure 1: An illustration of a computation tree of the LEVIATHAN stream cipher with height four.

```

end if
end for
output  $c(s)$ 

```

A computation tree is illustrated in Figure 1.

The keystream is the concatenation of the outputs of all trees, with the trees ordered so that the roots of the trees are in ascending order. Keystream generation can be done efficiently (in order) by using a preorder traversal of each tree, starting at the root. In addition, an arbitrary segment of the keystream can be efficiently generated by ‘seeking’ to the initial output, then continuing with the preorder traversal. The preorder tree traversal method of keystream generation is not specified, as it is independent of the definition of the keystream.

2.1 The functions a , b , and c

LEVIATHAN uses the ‘node number’ of the binary tree to ensure that two distinct nodes cannot have the same state. This number is denoted as z , and counted as part of the state of node to which it corresponds. We compute this number in the conventional way: the node number of the root is defined to be one, and if the node number of a node is z , that of its left child is $2z$ and that of its right child is $2z + 1$.

The low-level functions of the cipher use conventional cryptographic primitives: substitution, rotation, and addition modulo 2^n . The ‘filter’ function c is defined as the linear reduction of n bits to $n/2$ bits. The branching functions a and b are defined as the composition of a diffusion function d with the nonlinear ‘confusion’ functions f and g , respectively: $a = f \circ d$ and $b = g \circ d$. The precise definitions of these functions are

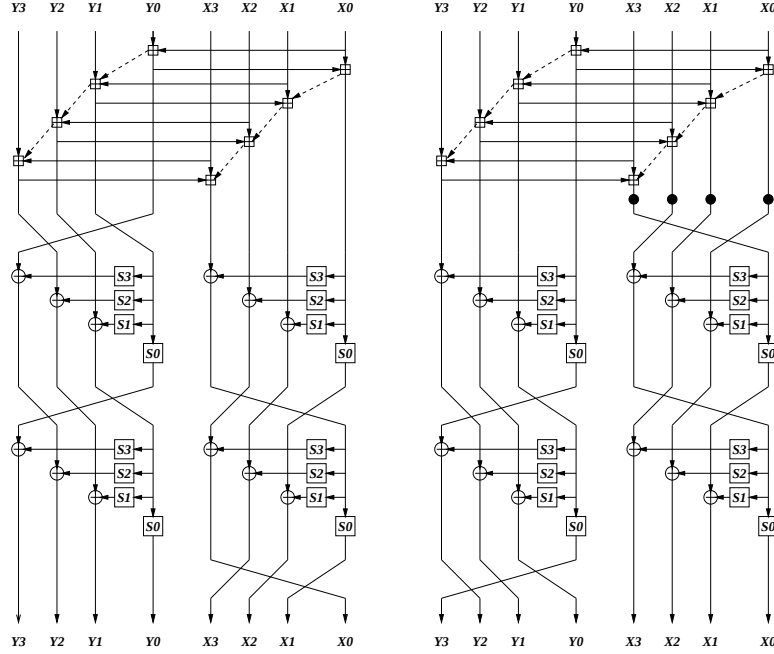


Figure 2: The functions $f \circ d$ (left) and $g \circ d$ (right). Here and below $\oplus$ denotes the bitwise exclusive or operation, $\boxplus$ denotes integer addition (with dashed lines indicating the carry operations), and $\cdot$ indicates the bitwise complementation operation (or logical ‘not’).

$$\begin{aligned}
 f(z \mid y \mid x) &= 2z \mid S(R(S(R(y)))) \mid L(S(L(S(x)))) \\
 g(z \mid y \mid x) &= 2z + 1 \mid L(S(L(S(y)))) \mid S(R(S(R(\neg x)))) \\
 d(z \mid y \mid x) &= z \mid x + y + z \mid 2x + y + z \\
 c(z \mid y \mid x) &= x \oplus y,
 \end{aligned} \tag{1}$$

where integer addition modulo two is denoted as $+$, bitwise exclusive-or is denoted as $\oplus$, and bitwise complementation is denoted as $\neg$. The functions R and L indicate rotation by $n/4$ bits and to the right and left, respectively (where ‘right’ means in the direction of the least significant bit). The filter function c is a bitwise exclusive or of x and y . The nonlinear function S is implemented using a key-dependent substitution table, as described below.

In order to illustrate the detailed workings of f and g , we define $X0, X1, X2, X3$ to represent the bytes of x , where $X0$ is the least significant byte, $X1$ is the second least significant, and so on. We also use similarly decatenated versions of the variables y and z . The function S depends only on the lowest $n/4$ bits

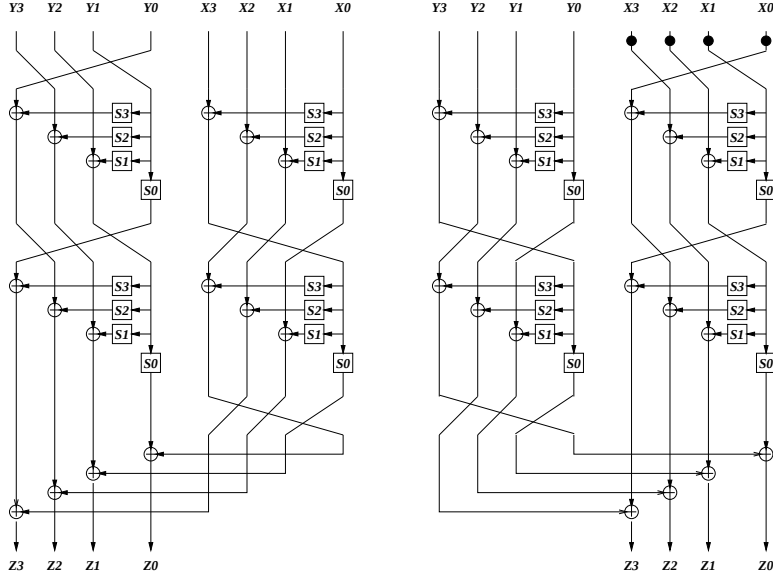


Figure 3: The functions $c \circ f$ and $c \circ g$, showing how the output of the cipher is derived from the state of the leaf nodes.

of its argument, and is defined using four component functions $S0, S1, S2, S3$, each of which maps $n/4$ bits to $n/4$ bits:

$$S(X3 \mid X2 \mid X1 \mid X0) = X3 \oplus S3(X0) \mid X2 \oplus S2(X0) \mid X1 \oplus S1(X0) \mid S0(X0).$$

The key setup is defined such that these component functions are invertible. The cipher can be analyzed as a computational graph, each node of which contains a single $n/4$ -bit byte, and this approach is followed in Figures 2-4.

The functions $a = f \circ d$ and $b = g \circ d$ are illustrated in Figure 2. The composition of c with f and g is shown in Figure 3.

2.2 Key Setup

A LEVIATHAN key consists of m bytes. Key setup consists of generating tables that store the four invertible functions, $S0, S1, S2$, and $S3$. The invertible functions are pseudorandomly generated by swapping elements at random in the table, in a key dependent manner. In the following, we define $q = 2^{n/4}$ to represent the number of elements in each table, we use the convention that a table with q elements is indexed from zero to $q - 1$. We let p represent the number of passes over the tables, and we specify a default value of two for this

parameter. The key setup algorithm takes a key K , considered as a string of m bytes, and is as follows:

```

for  $i$  from 0 to  $q$  do
  set  $S0[i]$  to 0
  set  $S1[i]$  to 0
  set  $S2[i]$  to 0
  set  $S3[i]$  to 0
end for
for  $j$  from 0 to 3 do
  for  $i$  from 0 to  $q$  do
    set  $S3[i]$  to  $S2[i]$ 
    set  $S2[i]$  to  $S1[i]$ 
    set  $S1[i]$  to  $S0[i]$ 
    set  $S0[i]$  to  $i$ 
  end for
  set  $k$  to  $j$ 
  for  $t$  from 1 to  $p$  do
    for  $i$  from 0 to  $q$  do
      set  $k$  to  $k + (K[i \bmod m] + S0[i]) \bmod q$ 
      exchange  $S0[i]$  with  $S0[k]$ 
      exchange  $S1[i]$  with  $S1[k]$ 
      exchange  $S2[i]$  with  $S2[k]$ 
      exchange  $S3[i]$  with  $S3[k]$ 
    end for
  end for
end for

```

3 Security

This section describes the security properties and expected security level of the cipher.

3.1 Statement on Hidden Weaknesses

The designers of LEVIATHAN are not aware of any weaknesses in the cipher, intentional or otherwise.

3.2 Security Analysis

The functions a and b are comparable to the round functions of a substitution-permutation network [7], and they are the main source of nonlinearity in the cipher. The function c can be used to hide the internal state, as it has fewer outputs than inputs.

Our security goals are that finding an unknown encryption key, given a length of keystream, should be as computationally expensive as an exhaustive search over the space of all keys. To this end, we analyze the indistinguishability of a segment of keystream from a truly random binary string, which is a sufficient condition to meet our goals. Below we enumerate important necessary conditions for indistinguishability.

3.2.1 Up and Down Attacks

The set $\{c(a(x)), c(b(x)) \mid x \in \mathcal{R}_s\}$, where $\mathcal{R}_s$ is a set of random n -bit elements, must be indistinguishable from a random set of $2n$ -bit elements. We call attacks on this property *up and down* attacks, and they are analogous to attacks on stream ciphers which use adjacent outputs.

The structures of f , g and c were heavily influenced by the requirement to resist up and down attacks. This structure is illustrated in Figure 3, which shows the concatenated view of $c \circ f$ and $c \circ g$. The variables at the top and bottom are two adjacent n -bit outputs of the cipher. The state at the middle of the network is the output of the function d for each of the leaves.

Up and down attacks based on differential cryptanalysis fail because of the filter function c : a differential does not provide a cryptanalyst with much information. A single byte difference in the middle of the concatenated view is diffused on its way out.

Up and down attacks based on linear cryptanalysis are more of a threat, and were be considered in more detail. Each eight-bit substitution table will have an expected bias of about 2^{-4} . Using the piling up lemma [2], we expect that the total bias for two adjacent outputs is 2^{-32} . Thus, about 2^{64} adjacent outputs are required in order to distinguish LEVIATHAN keystream from random keystream using a linear approach. As the number of outputs is 2^{48} , these attacks are avoided. Our numerical experiments support this analysis.

It is worth noting that the piling-up lemma applied to the function c (in which the right-hand side and the left-hand side are exclusive-ored together) is correct and precise. This is in contrast to its conventional usage in analyzing block ciphers, in which it is just an approximation.

3.2.2 Branch Attacks

The i^{th} leaf of each tree follows the same branch, and the outputs of those leaves are the result of applying the same function to distinct, known inputs. Each ‘branch function’ can be subjected to cryptanalytic attacks.

To analyze these attacks, we considered the set of *branch functions of length l* , defined by the composition of l functions chosen from the set $\{a, b\}$. The longest known differential of any branch function is of length three.

3.2.3 Top Down Attacks

The outputs corresponding to the leaf nodes of each tree must be indistinguishable from random. We call attacks on this property *top down* attacks, and they are analogous to known plaintext attacks on block ciphers. However, top down attacks are more difficult in that the function c hides part of the information. With a height $h = 16$, these attacks do not appear practical.

3.2.4 Slide Attacks

Each two distinct nodes must have distinct state. If two nodes at the same level have the same state, then the leaves that are descendants of those nodes will be pairwise identical. If two nodes at different levels have the same state, more sophisticated approaches can detect this condition. Such attacks are analogous to the ‘slide attacks’ on block ciphers [9, 10]. The use of the ‘node number’ z avoids these attacks.

4 Design Rationale

Unfortunately, many stream ciphers have a significant limitation: they are unable to efficiently seek to an arbitrary location in their keystream. Many applications require this capability. In a communications protocol that uses unreliable transport, data packets are not guaranteed to arrive in order, nor arrive at all. The IP, UDP, and RTP protocols fall into this category, and commonly experience loss and reorder in practice. A stream cipher can be used to provide privacy for that data if it can seek to the proper location in the keystream based on a sequence number. This approach forms the basis of the Stream Cipher ESP for the IP Security protocol [3]. Similarly, an encrypted disk partition or

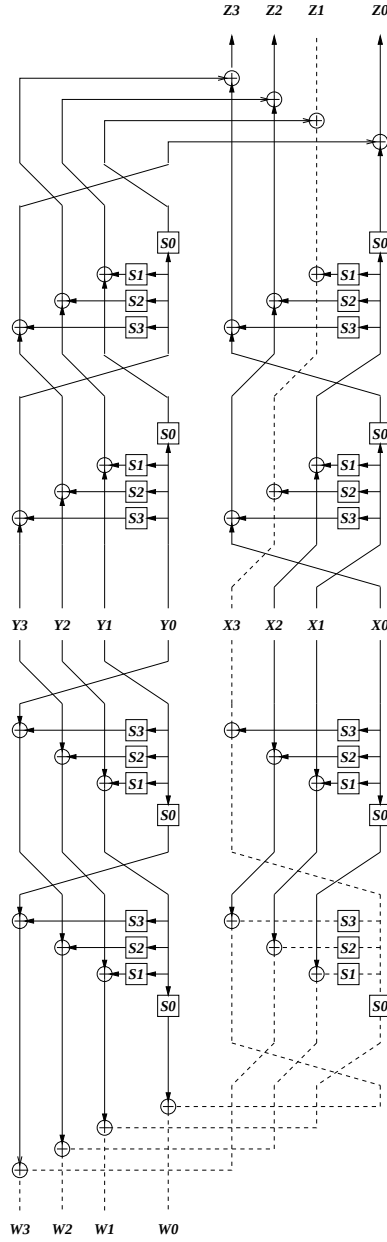


Figure 4: The concatenated view of $c \circ f$ and $c \circ g$. Here an example of the propagation of a byte-wise differential is shown, with dashed lines indicating the presence of a nonzero difference.

file system can use a stream cipher if that cipher supports the seek operation. These scenarios do not require the ‘random access’ capability of a block cipher, in which all inputs are equally easy to compute. Rather, they require the capability to seek into the keystream, with a seek time that is not significant relative to the time required to generate the keystream itself.¹

One design approach to stream ciphers that offers the seek capability is to have the state update function be linear in some field. In this case, a seek is a composition of linear operations, and is thus linear itself. This approach is similar to using a block cipher in counter mode [7], which imposes requirements on the output function that are similar to the requirements on block ciphers. We therefore turn to alternative approaches.

Another design approach is that taken by Rogaway and Coppersmith in the design of the SEAL cipher [5, 6]: define a special ‘seek function’ that pseudorandomly maps an index (and a fixed key) to an internal state of the cipher. From this state, a length of keystream is generated. The keystream for the cipher is defined to be the concatenation of the keystreams generated for each index, with indices in ascending order. Effectively, this approach creates a stream cipher that can seek to some regularly spaced locations in its keystream.

The SEAL approach is satisfactory for many applications. However, some applications may require the ability to seek to an arbitrary location in the keystream. For example, an encrypted database containing many small records could have this requirement. In addition, the seek function approach adds security requirements. The seek function itself must be secure, and the seek and advance functions must be such that they do not interact in an insecure way.

The use of a tree architecture for LEVIATHAN realizes the seek capability without using a linear state update function, and without a special seek function.

We required that the cipher achieve a high throughput on modern general purpose processors. This requires the cipher to work well with machines that have no more than seven available registers.

We required that the cipher be parameterizable, both in terms of the number of bits of state per node, and in terms of key size. The parameterizable width enables reduced size versions of the cipher to be studied, enabling more meaningful numerical experiments on the cipher. The parameterizable key size is useful to adapt the cipher to different security policies, and to increase the key size to protect against typical plaintext attacks [4].

For 32 bit processors, $n = 32$ is a natural parameter selection. For the IPSEC application [3], a cipher must be able to generate 2^{32} keystream segments of

¹We use ‘seek’ in the same sense as the comparable POSIX and ANSI C functions for repositioning the offset of a file descriptor or stream.

length 2^{16} bytes. This motivates the selection of parameters $h = 16$. The security requirement limits the number of outputs.

The detailed design of the functions a , b and c were discussed in the previous section. Here we note that the bit complementation operation was added to g , but is absent from f , in order to ‘break the symmetry’ of the concatenated view of $c \circ f$ and $c \circ g$.

5 Performance

Keystream generation (in order) is done by a preorder traversal of the tree, starting at the root. A binary tree with height h contains $2^h - 1$ nodes, 2^{h-1} leaves, and $2^{h-1} - 2$ edges (where we use the convention that a tree with one node has height one). Each edge is computed exactly once in a traversal of the tree. Therefore, the rate at which the cipher can generate keystream is approximately

$$\frac{r2^{h-1}}{R_c 2^{h-1} + R_{ab}(2^h - 2)} \simeq \frac{r}{R_c + 2R_{ab}} \quad (2)$$

where R_c is the rate at which the function c can be evaluated, and R_{ab} is the average rate at which the functions a and b can be evaluated.

The preorder traversal of the computation tree can be efficiently implemented by maintaining a stack of h elements, the i^{th} element of which contains the state information of the i^{th} node on the path from the root to the leaf corresponding to the current output. We think of the computation tree as defining a cipher with nh bits of internal state, the state update function of which modifies only a portion the current state. An important implementation consideration is that the preorder traversal is highly parallelizable.

The functions f and g can each be computed in seven instructions on a general purpose CPU that can issue two instructions simultaneously, using five registers (holding x , y , z , a temporary variable, and a pointer to the substitution table). The theoretical encryption rate is thus approximately $(2 + 2 \times 7)/32 = 1/2$ cycles per bit of keystream (where two execution units are available).

Our ANSI C implementation of LEVIATHAN produced a throughput of 375 Megabits per second on a 500 Mhz Pentium II processor. This is less than half of the speed that is theoretically achievable on a CPU with two execution units. The cause of this performance degradation is believed to be the failure of the C compiler to work around the paucity of registers available on that CPU. It is expected that an assembler implementation will produce better results.

6 Implementation Notes

Efficient implementation requires the use of a preorder tree traversal algorithm, preferably a non-recursive implementation. A good description of such an algorithm is provided by Sedgwick [8], and an illustration of its application to this cipher is provided in the reference code.

There are no known implementation weaknesses that are unique to LEVIATHAN.

On a processor with a word size of n bits, the nonlinear function S can be implemented using a single table lookup and a single exclusive-or operation. To do this, a table T is constructed using the following algorithm:

```
for  $i$  from 0 to  $q$  do
  set  $T[i]$  to  $S3[i] \mid S2[i] \mid S1[i] \mid S0[i] \oplus i$ 
end for
```

Then $S(x) = T[x \wedge 255] \oplus x$, where $\wedge$ is the logical ‘and’ operation, which serves here to mask off the lower $n/4$ bits.

The substitution operation LEVIATHAN maps a single n -bit variable x to $x \oplus T(x)$, where the substitution table T that depends only on the lowest 8 bits of x . Thus, invertibility requires that the function mapping the lowest 8 bits of x to lowest 8 bits of $x \oplus T(x)$ be invertible. This is achieved by pseudorandomly selecting an invertible function mapping 8 bits to 8 bits, then composing that function with the bitwise exclusive-or function.

References

- [1] Handschuh, H., and Gilbert, H., “Chi squared Cryptanalysis of the SEAL Encryption Algorithm”, Proceedings of the IVth Workshop on Fast Software Encryption, Lecture Notes in Computer Science, Springer Verlag.
- [2] Matsui, M., “Linear Cryptanalysis Method for DES Cipher”, Proceedings of Eurocrypt '93, Springer Verlag, 1993.
- [3] McGrew, D., Fluhrer, S., Peyravin, M., “The Stream Cipher Encapsulating Security Payload,” draft-mcgrew-ipsec-scesp-01.txt, Internet Draft, July, 2000.
- [4] McGrew, D., and Fluhrer, S., “Attacks on Linearly Redundant Plaintext and Implications on Internet Security”, Seventh Annual Workshop on Selected Areas in Cryptography, August, 2000.

- [5] Rogaway, P. and Coppersmith, D., "A Software-Optimized Encryption Algorithm", Proceedings of the 1994 Fast Software Encryption Workshop, Lecture Notes In Computer Science, Volume 809, Springer-Verlag, 1994, pp. 56-63.
- [6] Rogaway, P. and Coppersmith, D., "A Software-Optimized Encryption Algorithm", Journal of Cryptology, Volume 11, Number 4, Springer-Verlag, 1998, Pages 273-287. Also available on the Internet at <http://www.cs.ucdavis.edu/~rogaway/papers/seal-abstract.html>.
- [7] Schneier, B. "Applied Cryptography: Protocols, Algorithms, and Source Code in C", Wiley, 1996.
- [8] Sedgwick, R., "Algorithms in C", Addison-Wesley, 1992.
- [9] Biryukov, A. and Wagner, .D, "Slide Attacks" Proceedings of the Fast Software Encryption Workshop, Springer-Verlag, 1999.
- [10] Biryukov, A. and Wagner, .D, "Advanced Slide Attacks" Proceedings of EUROCRYPT 2000, Springer-Verlag, 2000.

7 Test Vectors

This section presents several outputs for specific keys, for use as ‘test vectors’ to validate implementations. In all cases, the parameters specified above have been used. The first eight words of output are given as hexadecimal integers, and the keys are specified as strings of hexadecimal numbers.

key: 0xde, 0xc0, 0xc0, 0xc5, 0x57, 0xfa

keystream: 0x1861600e, 0x88244832, 0x2a6d8201, 0xffd0f37d, 0xb8767ce6, 0xe7bd8954, 0xb3fc97f0, 0xe88caba1

key: 0x01, 0x23, 0x45, 0x67, 0x89, 0xab, 0xcd, 0xef 0xfe, 0xca, 0xbe, 0xba, 0xbe, 0xba, 0xfe, 0xca

keystream: 0x44a7742e, 0xba1625e3, 0x1c00e70a, 0x71fffd3b, 0x9aa8ea1f, 0xd02a5f08, 0xef52ffd7, 0x8e3b31e7

key: 0x01, 0x23, 0x45, 0x67, 0x89, 0xab, 0xcd, 0xef, 0xfe, 0xca, 0xbe, 0xba, 0xbe, 0xba, 0xfe, 0xca, 0xbe, 0xba, 0xfe, 0xca, 0xfe, 0xca, 0xbe, 0xba, 0x89, 0xab, 0xcd, 0xef, 0x01, 0x23, 0x45, 0x67

keystream: 0x669017c4, 0xb85204eb, 0x25aab14a, 0x99cde76b, 0x163c76bb, 0x6c7b12a6, 0xd5c8c1e8, 0xf550c41c